

RELATÓRIO TÉCNICO

Integração de Llama local ao pipeline de verificação

Backend de inferência reproduzível via llama.cpp server, preservando Gemini para auditoria histórica e preparando migração futura para vLLM.

ESCOPO	RESULTADO
Arquitetura	Provider-neutral com adapter OpenAI-compatible
Execução inicial	CPU-only, 7B/8B GGUF Q4_K_M, concorrência 1
Validação	177 testes passando; zero falhas e zero skips
Data	27/09/2026

Projeto: llm_verification_pipeline_gemini

Status: implementação concluída; smoke real aguarda servidor e GGUF local

1. Sumário executivo

Foi criada uma camada de providers que permite executar Stage 1 e Stage 2 com um servidor Llama externo compatível com a API de chat da OpenAI. O primeiro alvo é o llama.cpp server em CPU, mas o contrato também permite apontar o mesmo provider para vLLM sem reescrever retrieval, stages, agregação ou avaliação.

Conclusão

A infraestrutura está funcional e coberta por testes mockados end-to-end. O pipeline local preserva os schemas públicos, checkpoints, artefatos e regras existentes. Nenhuma inferência Gemini foi realizada durante a implementação.

177

testes passando

18

testes novos

0

falhas

12

issues no smoke

Objetivos atendidos

- Provider Gemini preservado para reprodução histórica e comparação.
- Provider OpenAI-compatible implementado para llama.cpp e futuro vLLM.
- JSON estruturado validado estritamente; erros nunca viram decisão negativa silenciosa.
- Retries limitados, telemetria por tentativa e resume conservador por fingerprint.
- Servidor separado do pipeline, sem download ou inicialização automática.
- Amostra determinística de 12 issues com casos simples e difíceis.

Fora de escopo

Não foram modificados prompts, critérios semânticos, catálogo, retrieval, thresholds ou agregação. Também não foram corrigidos os falsos positivos conhecidos do Stage 2, executados os 200 casos completos, baixados modelos, alugadas GPUs ou processados os 391 mil registros.

2. Arquitetura

Estado anterior

Os requests já eram montados em uma estrutura reutilizável, mas client.py criava diretamente o SDK google-genai, interpretava respostas Gemini, fazia retries e expunha o Batch API. A telemetria lia usage_metadata e os manifests persistiam provider/família de API fixos. Checkpoints eram neutros, porém não validavam provider nem fingerprint do modelo.

Estado atual

```
Stage 1 / Stage 2
-> request canônico + JSON Schema público existente
-> provider selecionado
Gemini: google-genai / generateContent
OpenAI-compatible: POST /v1/chat/completions
-> resposta canônica
-> parsing, normalização, checkpoint e agregação existentes
```

Componente	Responsabilidade
ProviderConfig	Seleciona backend, endpoint, modelo, structured output, geração, provenance e capabilities.
ProviderCapabilities	Declara suporte a sync, remote batch, JSON Schema, JSON object, usage, seed e contagem de tokens.
OpenAICompatibleClient	Traduz requests canônicos para HTTP e devolve o formato consumido pelos stages existentes.
client.py	Seleciona provider, mantém parsing canônico, política de retry e caminho Gemini.
artifacts.py	Persiste fingerprints, modelo, servidor, geração, hashes e configuração sem secrets.
telemetry.py	Registra tentativas, token source, latência, parsing, erros e throughput.

Compatibilidade

- Gemini continua com o comportamento e Batch API existentes.
- Local/OpenAI-compatible suporta apenas sync; --mode batch falha explicitamente.
- O alias local é normalizado para openai_compatible nos metadados.
- Runs Gemini históricos com provider google-gemini continuam reconhecidos no resume.

3. Structured output, parsing e retries

A integração prioriza constrained decoding por JSON Schema. A capability é explícita porque servidores OpenAI-compatible não implementam necessariamente o mesmo `response_format`.

Modo	Uso	Comportamento
json_schema	Preferencial	Envia JSON Schema nativo; dialect llama_cpp ou openai.
json_object	Fallback explícito	Solicita objeto JSON e acrescenta o schema como instrução técnica.
prompt_only	Último recurso	Sem <code>response_format</code> ; parser e schema continuam obrigatórios.

Falhas tratadas

Categoria	Tratamento
JSON inválido ou texto extra	Rejeitado; retry limitado; erro persistido.
Resposta vazia	Rejeitada; nunca interpretada como ausência de pattern.
Campo ausente / enum inválido	Rejeitado pela validação estrutural ou normalização semântica existente.
Resposta truncada	finish_reason diferente de stop vira resposta incompleta e retryável.
Timeout / conexão / reset	Erro de transporte retryável com backoff e limite configurado.
HTTP 4xx/5xx	Classificado por status; autenticação não repete, falhas transitórias podem repetir.
Model error	Erro explícito e auditável, sem alteração do conteúdo semântico.

Invariante de pesquisa

Uma falha técnica ou estrutural permanece erro retomável. Ela não é convertida em `verdict=no` e não altera os critérios metodológicos.

Determinismo

Os defaults locais são `temperature=0` e `seed=42`. `top_p`, `max_tokens` e `stop sequences` também são explícitos e persistidos. Isso melhora a repetibilidade, mas não garante determinismo absoluto entre builds, hardware, kernels ou versões do servidor.

4. Configuração e proveniência

Variável	Default	Finalidade
LLM_PROVIDER	gemini	gemini, local ou openai_compatible
LOCAL_LLM_BACKEND	llama.cpp	Rótulo do backend; use vllm futuramente
LOCAL_LLM_BASE_URL	http://127.0.0.1:8080	Raiz do servidor; /v1 final é normalizado
LOCAL_LLM_MODEL	local-model	Alias/modelo enviado ao servidor
LOCAL_LLM_API_KEY	unset	Bearer opcional; nunca persistido
LOCAL_LLM_TIMEOUT_SECONDS	300	Timeout por request
LOCAL_LLM_CONCURRENCY	1	Registrado; runner ainda serial
LOCAL_LLM_STRUCTURED_OUTPUT	json_schema	json_schema, json_object ou prompt_only
LOCAL_LLM_SCHEMA_DIALECT	backend	llama_cpp ou openai
LOCAL_LLM_TOP_P	1.0	Parâmetro de amostragem
LOCAL_LLM_STOP	unset	Sequências separadas por
LOCAL_LLM_CONTEXT_WINDOW	8192	Contexto registrado
LOCAL_LLM_MAX_ATTEMPTS	4	Tentativas máximas
LOCAL_LLM_COUNT_TOKENS_WHEN_MISSING	true	Usa tokenizer do servidor se usage faltar
LLAMA_MODEL_PATH	unset	Caminho GGUF e provenance
LOCAL_LLM_MODEL_SHA256	unset	Hash fornecido externamente
LOCAL_LLM_HASH_MODEL	true	Calcula SHA-256 uma vez por processo/run
LOCAL_LLM_QUANTIZATION	inferida	Override da quantização
LOCAL_LLM_COMPUTE_MODE	unknown	CPU/GPU provenance
LOCAL_LLM_SERVER_VERSION	unset	Versão opcional
LOCAL_LLM_SERVER_COMMIT	unset	Commit opcional

Segurança

LOCAL_LLM_API_KEY é removida dos metadados. URLs com usuário ou senha embutidos são rejeitadas. O backend local acessa somente o endpoint configurado.

5. Operação do llama.cpp

Terminal 1 - servidor CPU-only

```
export LLAMA_MODEL_PATH=/absolute/path/Meta-Llama-3.1-8B-Instruct-Q4_K_M.gguf
export LLAMA_MODEL_ALIAS=local-model
export LLAMA_CONTEXT_SIZE=8192
export LLAMA_THREADS=4
export LLAMA_GPU_LAYERS=0
export LLAMA_PARALLEL=1
./scripts/start_llama_server.sh
```

O script não assume CUDA, usa uma única slot paralela e mantém o lifecycle do servidor sob controle do operador. Também aceita LLAMA_SERVER_BIN, LLAMA_HOST e LLAMA_PORT.

Verificação manual de conectividade

```
export LLM_PROVIDER=local
export LOCAL_LLM_BASE_URL=http://127.0.0.1:8080
export LOCAL_LLM_MODEL=local-model
PYTHONPATH=. .venv/bin/python scripts/check_local_llm.py
```

O comando consulta health/model metadata e envia apenas um pequeno request estruturado. Não faz parte do pytest padrão.

Terminal 2 - primeiro smoke local

```
export LLM_PROVIDER=local
export LOCAL_LLM_BACKEND=llama.cpp
export LOCAL_LLM_BASE_URL=http://127.0.0.1:8080
export LOCAL_LLM_MODEL=local-model
export LLAMA_MODEL_PATH=/absolute/path/Meta-Llama-3.1-8B-Instruct-Q4_K_M.gguf
export LOCAL_LLM_STRUCTURED_OUTPUT=json_schema
export LOCAL_LLM_SCHEMA_DIALECT=llama_cpp
export LOCAL_LLM_CONTEXT_WINDOW=8192
export LOCAL_LLM_CONCURRENCY=1

RETRIEVAL_CONFIDENCE_THRESHOLD=0.35 \
RETRIEVAL_REQUIRE_LEXICAL_ANCHOR=false \
SEMANTIC_BACKEND=lsa \
PYTHONPATH=. .venv/bin/python run_pipeline.py run \
--input data/smoke/llama_local_smoke.csv \
--provider local --profile optimized --mode sync \
--temperature 0 --seed 42 --run-id llama_local_smoke_v1
```

6. Resume, contexto e telemetria

Checkpoint e resume

Ctrl-C preserva resultados concluídos. Reexecutar o mesmo comando e run-id ignora Stage 1 e Stage 2 já concluídos e tenta novamente registros pendentes/recuperáveis. Mudanças de provider, endpoint, geração, contexto, caminho/tamanho/hash do modelo ou demais campos do fingerprint rejeitam o resume.

```
PYTHONPATH=. .venv/bin/python run_pipeline.py status \
--run-dir outputs/runs/llama_local_smoke_v1
```

Orçamento de contexto observado

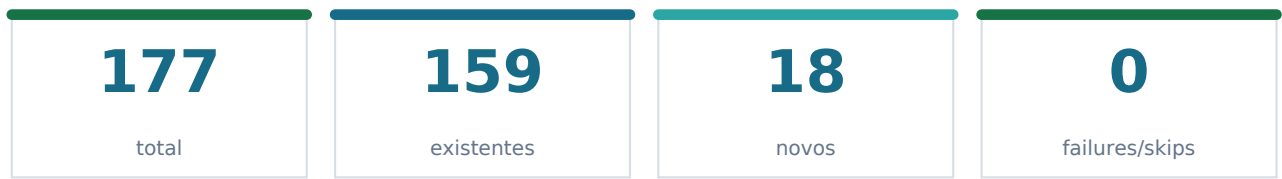
Request	Máximo de caracteres	chars/4 diagnóstico
Stage 1 optimized	22.554	5.639
Stage 2 optimized	21.175	5.294
Output configurado	-	até 2.048 tokens

A divisão por quatro não é tokenização oficial. O uso reportado pelo provider é autoritativo; quando ausente, o endpoint de tokenização do próprio servidor pode preencher input tokens com source=server_tokenized. Uma janela de 8.192 é o mínimo prático e pode ficar apertada; 12.288 ou 16.384 é preferível quando a RAM permitir.

Artefatos

Arquivo	Conteúdo
run_metadata.json	Provider/model fingerprint, geração, hashes, quantização, contexto e servidor.
provider_runtime.json	Respostas de /health e /v1/models capturadas por processo.
token_calls.jsonl	Cada tentativa: tokens, source, latência, parsing, stop reason e erro.
token_summary.json	Resumo comum a todos os providers.
local_llm_summary.json	Chamadas lógicas, falhas finais, retries, p50/p95 e throughput local.
stage1/stage2/issue_results.csv	Contratos públicos existentes, sem duplicação por provider.

7. Validação



Cobertura nova

Área	Cobertura
Seleção/configuração	Gemini, local, base URL, ausência de API key, credentials rejeitadas.
Requests	Stage 1, Stage 2, JSON Schema llama.cpp e preservação do schema público.
Parsing	JSON válido, inválido, vazio, texto extra, campo obrigatório e truncamento.
Transporte	HTTP 500, timeout, connection refused, model error e retry.
Usage	Provider usage presente, ausente e fallback pelo tokenizer do servidor.
Proveniência	Probe de modelo, arquivo GGUF, tamanho, quantização e SHA-256.
Resume	Skip de item concluído, fingerprint/model/provider incompatíveis.
End-to-end mockado	Optimized Stage 1 + Stage 2 + agregação + metadata + telemetria + resume.

Amostra de smoke

A amostra determinística contém 12 issues do conjunto de 200 casos, preservando as anotações originais. Ela combina positivos e negativos humanos, casos simples e difíceis, background architecture, false friends, discussão insuficiente e Time-Constrained Access. Serve para integração e debugging, não para estimativa final do artigo.

Resultado desta execução
O dry-run local produziu 12 requests e o status mostrou os 12 itens pendentes sem acessar servidor ou Gemini. O smoke real não foi executado porque llama-server e um GGUF não estão instalados nesta máquina.

8. Limitações e próximos passos

Limitações atuais

- Uma inferência 7B/8B Q4 em CPU será lenta e adequada somente a smoke/debugging.
- LOCAL_LLM_CONCURRENCY é persistida, mas o runner local permanece serial.
- Remote batch não é suportado pelo provider OpenAI-compatible nesta etapa.
- Structured output depende das capabilities efetivas e do dialect configurado no servidor.
- Seed e temperature=0 não garantem determinismo absoluto entre versões e hardware.
- Os 200 registros existentes são development/reference set, não avaliação final independente.

Migração futura para vLLM

```
export LLM_PROVIDER=openai_compatible
export LOCAL_LLM_BACKEND=vllm
export LOCAL_LLM_BASE_URL=http://gpu-host:8000
export LOCAL_LLM_MODEL=your-served-model-name
export LOCAL_LLM_SCHEMA_DIALECT=openai
```

Sequência recomendada

Etapa	Ação	Gate
1	Executar scripts/check_local_llm.py	Health, model info, schema e usage válidos
2	Executar o smoke de 12 casos	Sem erros estruturais ou de resume
3	Revisar outputs manualmente	Comportamento plausível nos casos difíceis
4	Rodar pequena amostra de desenvolvimento	Desempenho e qualidade aceitáveis
5	Ajustar semanticamente o Stage 2	Somente após integração aprovada

Parecer final

A mudança de backend foi isolada da metodologia. A base está pronta para um primeiro smoke real com llama.cpp e para posterior execução em vLLM/GPU, preservando auditabilidade e comparabilidade com os runs históricos.

Referência técnica

llama.cpp server README: <https://github.com/ggml-org/llama.cpp/blob/master/tools/server/README.md>